1. The PhoneGap support will be added by adding *cordova.js* in a *<script>* tag.
2. The *document.addEventListener* function will let PhoneGap load.
3. When PhoneGap is loaded, the openDatabase() function will create a new database with the details provided, i.e., database name, version, etc. The size of the database is 1,000,000 bytes.
4. After creating and opening the database, the database will call the transaction, which will call the insertDB method, and if the insertion is successful, then successDB will be called; else, errorDB will be called.
5. In the insertDB method, if the table 'CountryList' already exists then the table will be deleted.
6. This creates a new table 'CountryList', with two columns—*countrycode* and *name*.
7. This inserts the values for the *countrycode* and *name* in 'CountryList'.
8. If the values are successfully inserted in the table, it displays an alert 'success' and selects all the rows to display.
9. and
10. This displays the number of rows of data returned in the console.

Note: *In this example, the values returned are displayed only in the console, but in real time, you can show the values in different formats, i.e., list view, grid view, etc, according to your requirements.*

Media

This API gives the ability to play and record audio on the device. The different ways in which you can use the Media API are:

1. media.getCurrentPosition(): Returns the current position within the audio file.
2. media.getDuration(): Returns the duration of the audio file.
3. media.play(): Will start playing the audio file. Can be used to resume a paused file.
4. media.pause(): Will pause a playing audio file.
5. media.release(): Will release all the OS's audio resources being used by the file.
6. media.seekTo(): Will move the position to a certain point in the audio file.
7. media.startRecord(): Will start recording an audio file.
8. media.stopRecord(): This will stop recording an audio file.
9. media.stop(): This will stop the playing audio.

The arguments to pass through are:

1. src: The path of the audio file.
2. mediaSuccess: The callback to be called when a media object has played/paused or stopped the audio successfully.
3. mediaError: The callback returned when failure occurs within the process.
4. MediaStatus: The callback that is invoked to indicate the status changes.

```
//media.html
<html>  <head> <title>Media Example</title>
   <script type="text/javascript” charset=”utf-8”
src=”cordova-2.7.0.js”></script>
 <script type=”text/javascript” charset=”utf-8”>
  var recording;
  var recInterval;
        document.addEventListener(“deviceready”,
onDeviceReady, false);

        function onDeviceReady() (
        }
       var audio = null;
        var timer = null;


        function playAudio(src) {
          audio = new Media(src, onSuccess, onError);   (1)
           audio.play();     (2)
       if (timer == null) {
                timer = setInterval(function() {    (3)
                       audio.getCurrentPosition(

                      function(position) {
                            if (position > -1) {
                                setAudioPosition((position) +
“ sec”);   (4)
                            }
                        },

                        function(e) {
                            console.log(“Error occured=” +
e);
                            setAudioPosition(“Error: “ + e);
                        }
                    );
                }, 1000);
            }
        }
        function pauseAudio() {
            if (audio) {
                audio.pause();      (5)
            }
        }


        function stopAudio() {
            if (audio) {
                audio.stop();    (6)
```